

Sommaire

Projet Pokédex — Documentation technique	1
1. Présentation.....	1
2. Objectifs	1
3. Stack technique.....	2
4. Structure du projet.....	2
5. Fonctionnalités prévues	2
6. Logique technique — points clés.....	3
7. Difficultés rencontrées et retour d'expérience.....	3
8. Compétences BTS SIO mises en œuvre.....	4
9. Bilan et améliorations possibles	4
10. Liens utiles	5

Projet Pokédex — Documentation technique

Auteur : William TOUSSON · BTS SIO SLAM 1^{re} année · IPSSI SQY **Période :** 04/02/2025 – 08/02/2025 **Type :** TP individuel — Réalisation en cours de formation **Dépôt :** <https://github.com/william-tsn/pokedex>

Important : l'API utilisée par ce projet (**PokéBuildAPI** — pokebuildapi.fr) **n'est plus accessible** au moment de la rédaction de cette documentation. Le code reste fonctionnel mais l'application ne peut plus afficher de Pokémon car la source de données externe est désactivée. Ce point est analysé en section 7 (*Difficultés*) et constitue un retour d'expérience à part entière sur la dépendance aux services tiers.

1. Présentation

Application web statique de type **Pokédex interactif** composée de trois pages HTML alimentées par une API REST publique de Pokémon. Le projet a été réalisé sans framework et sans étape de build : le code s'exécute directement dans le navigateur.

2. Objectifs

- Mettre en pratique les bases du **JavaScript** (manipulation du DOM, fetch).
- Consommer une **API REST externe** pour afficher des données dynamiques.
- Utiliser **Tailwind CSS** pour le design et la responsivité.
- Réaliser une interface **multi-pages** cohérente avec navigation.

3. Stack technique

Composant	Détail
HTML5	3 pages : accueil.html, pokedex.html, liste.html
JavaScript Vanilla	Aucun framework — fichiers index.js et liste.js
Tailwind CSS v4	Chargé via CDN (@tailwindcss/browser@4)
CSS personnalisé	style.css (animations, grille, scrollbar masquée)
API REST	PokéBuildAPI — https://pokebuildapi.fr/api/v1/pokemon (désactivée depuis)
Hébergement	Aucun (statique, ouvrable en local)

4. Structure du projet

pokedex/	
├─ accueil.html	← Page d'accueil
├─ pokedex.html	← Pokédex interactif (1 Pokémon à la fois)
├─ liste.html	← Liste complète des Pokémon
├─ index.js	← Logique du Pokédex + menu + fond animé
├─ liste.js	← Logique de la grille complète
├─ style.css	← Styles personnalisés
├─ assets/	← Images (logo, Pokéball, energy.png...)

5. Fonctionnalités prévues

5.1 Page d'accueil (accueil.html)

- Navigation responsive (menu hamburger sous la largeur md).
- Présentation des deux modes (Pokédex et Liste).
- Fond décoratif avec Pokéballs positionnées aléatoirement.

5.2 Page Pokédex (pokedex.html)

- Affichage d'**un seul Pokémon à la fois** dans l'image du Pokédex.
- Clic sur l'image → Pokémon suivant (la navigation reboucle).
- Couleur de la carte selon le(s) type(s) : **dégradé bi-type** si applicable.
- Affichage : sprite, nom, icônes de types, identifiant #id.

5.3 Page Liste (liste.html)

- Récupération de **tous les Pokémon** depuis l'API.
- Disposition en grille de **4 colonnes**.
- Carte avec couleur de fond selon le type principal.

- Effet *hover* : passage en jaune + zoom léger.
-

6. Logique technique — points clés

6.1 Appel à l'API

```
fetch("https://pokebuildapi.fr/api/v1/pokemon")
  .then((res) => res.json())
  .then((data) => {
    pokemonData = data;
    displayPokemon(currentIndex);
  })
  .catch((err) => console.log(err));
```

L'API renvoyait un tableau JSON contenant pour chaque Pokémon : id, name, image, apiTypes.

6.2 Affichage dynamique d'une carte

```
if (secondaryType) {
  card.style.background = `linear-gradient(to right,
    ${typeColors[primaryType]} 50%,
    ${typeColors[secondaryType]} 50%)`;
} else {
  card.style.backgroundColor = typeColors[primaryType];
}
```

Génération d'un dégradé CSS dynamique en JavaScript selon les types du Pokémon. La table `typeColors` mappe les 18 types Pokémon à des couleurs hexadécimales.

6.3 Navigation circulaire

```
currentIndex = (currentIndex + 1) % pokemonData.length;
```

Le **modulo** permet de revenir au premier Pokémon après le dernier — astuce simple pour une navigation infinie.

7. Difficultés rencontrées et retour d'expérience

7.1 Difficultés techniques pendant le développement

- Découverte des Promises et de `fetch` (premier projet API).
- Gestion des Pokémon **bi-types** : il a fallu différencier les cas mono / bi-type pour générer le bon background.
- Mise en place de Tailwind via CDN.
- Adaptation responsive (menu hamburger).

7.2 Limite identifiée : la dépendance à une API tierce

Le projet repose sur une API REST hébergée par un tiers (PokéBuildAPI). Cette API a depuis été **désactivée** : aujourd'hui, la requête GET `https://pokebuildapi.fr/api/v1/pokemon` renvoie un code **HTTP 403** (accès interdit), et l'application ne peut plus afficher de Pokémon.

Vérification effectuée : test direct sur le domaine, code 403 confirmé.

Enseignement tiré : un projet web qui dépend d'un service externe est exposé au **risque de discontinuité** dès que ce service est arrêté, modifié ou rendu payant. Pour un projet pérenne, plusieurs solutions existent :

- Utiliser une API **plus stable et reconnue** (ex. [PokéAPI](#), la référence du domaine, maintenue depuis 2014).
- Mettre en place un **cache local** des données récupérées (fichier JSON statique).
- Prévoir un **mode dégradé** côté utilisateur lorsque l'API ne répond pas.

C'est précisément la sous-compétence du référentiel : « *Vérifier les conditions de la continuité d'un service informatique* ».

8. Compétences BTS SIO mises en œuvre

Ce projet justifie les compétences suivantes du **Bloc 1** :

- **C3 — Développer la présence en ligne de l'organisation** → Sous-compétence : « *Participer à l'évolution d'un site Web exploitant les données de l'organisation* ». Le site consomme une API REST externe et transforme ses données en composants visuels (cartes typées, dégradés bi-type).
 - **C6 — Organiser son développement professionnel** → Sous-compétence : « *Mettre en place son environnement d'apprentissage personnel* ». Le projet a été l'occasion d'auto-apprendre Tailwind CSS v4 et de pratiquer les Promises JavaScript.
-

9. Bilan et améliorations possibles

Ce que le projet m'a apporté :

- Compréhension du cycle complet *requête* → *réponse* → *affichage* avec une API REST.
- Pratique de Tailwind CSS et de la responsivité.
- Manipulation avancée du DOM (création d'éléments, classes dynamiques).
- Prise de conscience des **enjeux de dépendance** à un service externe (cf. §7.2).

Pistes d'amélioration identifiées :

- **Migration vers PokéAPI** (pokeapi.co) pour rendre le projet de nouveau fonctionnel et plus pérenne.
 - Ajout d'une **barre de recherche** (par nom ou par type).
 - **Pagination** sur la page Liste (actuellement, tous les Pokémon sont chargés d'un coup).
 - Gestion d'**erreur utilisateur** en cas d'échec de l'API (actuellement, simple `console.log` sans message visible).
 - Installation de **Tailwind en local** plutôt que via CDN (recommandation officielle de Tailwind pour la production).
-

10. Liens utiles

- Dépôt GitHub du projet : <https://github.com/william-tsn/pokedex>
- Documentation Tailwind CSS : <https://tailwindcss.com/docs>
- PokéAPI (alternative pérenne, à privilégier) : <https://pokeapi.co/>